Understanding GPU Resource Interference One Level Deeper

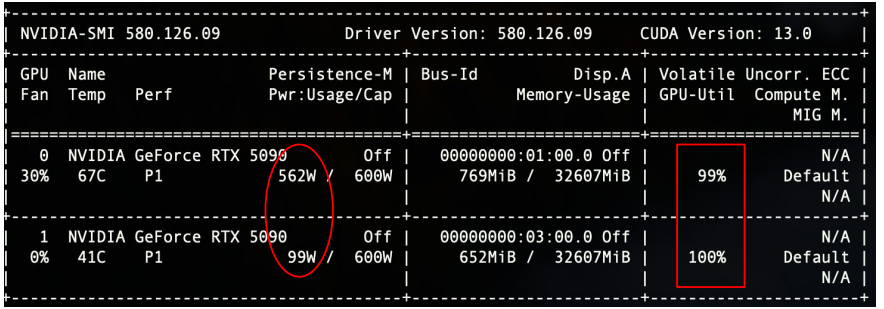
Paper Review

Recap on GPU Utilization

Key Insight:

The widely used nvidia-smi tool does not provide a comprehensive measure of GPU utilization comparable to CPU utilization.

A reported utilization of 100% does not necessarily mean that the GPU is fully saturated.

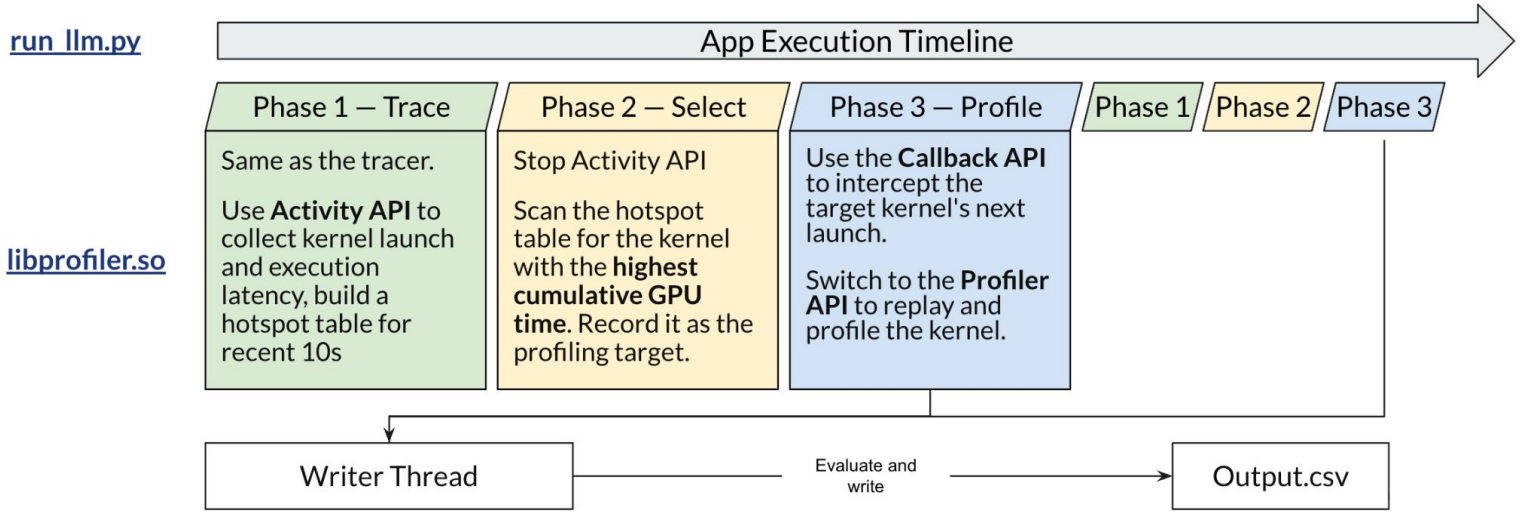


We explored a variety of tools to measure GPU utilization from **basic device monitoring** to **detailed kernel analysis** and **custom production instrumentation**.

Tool	Capabilities	Typical Use	Overhead
nvidia-smi / NVML	Reports **device utilization, power consumption, memory usage, and clock speeds**	Quickly confirm **GPU activity and operating status**	**Negligible**
DCGM	Provides **device-level telemetry**, such as SM activity, Tensor Core utilization, and DRAM bandwidth across clusters	Support continuous **cluster monitoring, dashboards, and alerts**	**Less than 1%**
Nsight Systems	Captures **CPU and GPU timelines**, including kernels, memory transfers, and API calls	Identify **launch gaps, stalls, synchronization issues, and serialization**	**Low, primarily during development**
Nsight Compute	Collects **detailed hardware counters** and performs **kernel roofline analysis**	Diagnose kernel bottlenecks and guide **targeted performance optimization**	**High due to replay**
CUPTI	Provides programmable APIs for **kernel tracing, API interception, and counter collection**	Build **custom production instrumentation** with minimal application changes	**Varies with the enabled features**

Recap on GPU Utilization

We also showed a customized kernel tracing/profiling tool developed using CUPTI's Activity API, Callback API and Profiler API

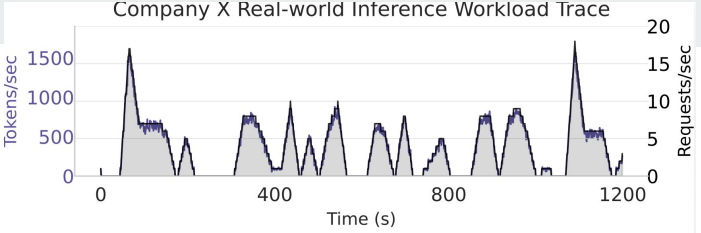


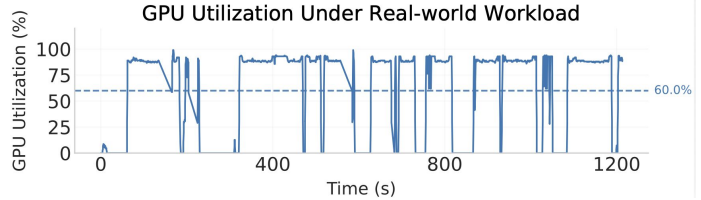
Kernel Colocation & Interference

How to improve GPU utilization?
Underutilized workloads can't use all GPU resources at once.

A GPU workload → a stream of GPU kernels.

Underutilization lives at the kernel level.
To reclaim it, one natural intuition is to colocate independent CUDA kernels







Is kernel colocation simply about filling unused resources?

Can kernels be fully isolated, or will shared resources cause interference?

GPU Architecture: Overview



GPU Architecture: Compute Units



GigaThread Engine:
Main scheduler that oversees the allocation of thread blocks to the entire GPU, including all GPCs.
Also enables Multi-instance GPU feature

Texture Processing Cluster (TPC):

Subunit of GPC. Schedule and distribute tasks to SMs. TPC has its own cache to coordinate data flow between SMs and global memory

Graphic Processing Cluster (GPC):

Highest-level computing unit. Manage the organization and distribution of tasks across lower level computing units (TPCs and SMs)

Streaming Multiprocessors (SMs):

The most basic computing units of GPU. Executing instructions in parallel using thousands of lightweight threads.
Detailed in next page.

Streaming Multiprocessors



Each SM contains:

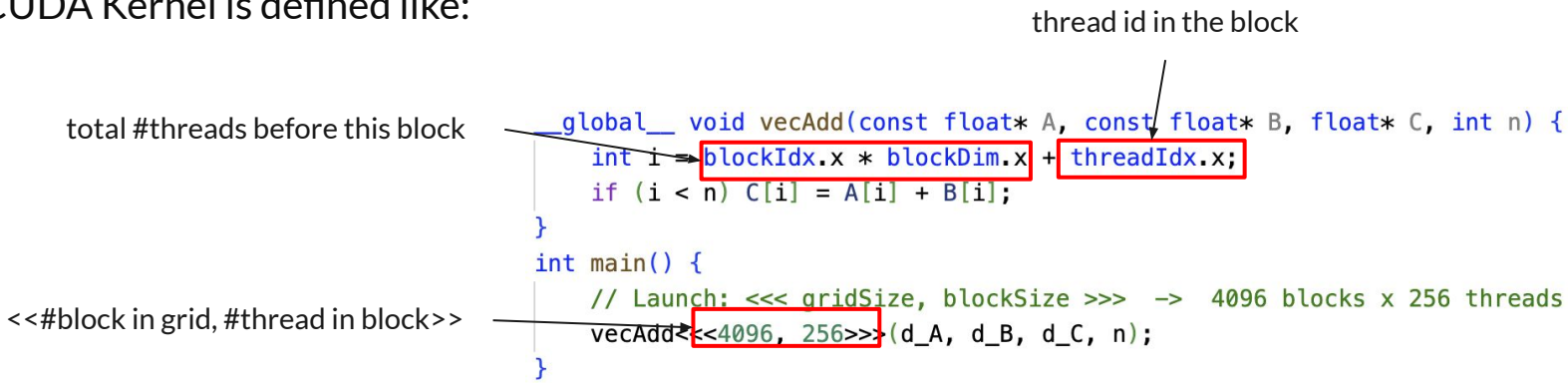
- **Tensor Memory Accelerator:** optimizes data transfers between memory hierarchies, supporting asynchronous copying of multi-dimensional data.
- **Shared Memory / L1 Cache:** configurable for cache and sharing, share data between SMs
- **And 4 SM Subpartition (SMSP), each contains**
  - **1 Warp Scheduler:** manage the execution of warps on SM
  - **32 CUDA Cores:** perform simple arithmetic operations
  - **1 Tensor Cores**: perform mixed-precision matrix and tensor operations
  - **16384 (16K) Registers:** fast, on-chip memory used to store variables for each thread.
  - **1 Special Function Units:** handle complex mathematical functions, such as sine, cosine, and square roots.

Programming Model vs GPU Hardware

GPU Hardware hierarchy:

GPU → **SMs** → **SMSP** (warp scheduler) → **Execution Pipelines** (CUDA cores, SFT, Tensor Core, etc)

A CUDA Kernel is defined like:



Programming model hierarchy:

Kernel (Grid) → **Thread Blocks** → (**Warps** →) **Threads**

How does the programing concept maps on the GPU?

Programming Model vs GPU Hardware

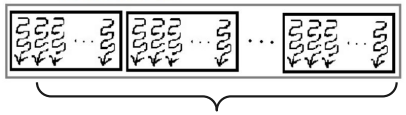
CUDA Kernel

```cuda
__global__ void vecAdd(const float* A, const float* B, float* C, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) C[i] = A[i] + B[i];
}
int main() {
    // Launch: <<< gridSize, blockSize >>>  ->  4096 blocks x 256 threads
    vecAdd<<<4096, 256>>>(d_A, d_B, d_C, n);
}
```



Send to GPU

Kernel Grid

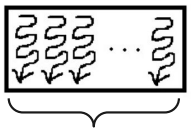


4096 Blocks
(defined by kernel)



Decomposed to Blocks, scheduled by the GPU Block Scheduler

Thread Block

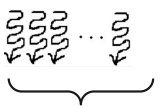


256 threads
(defined by kernel)



Assigned to one SM until finished

Warp



32 threads
(fixed)



Assigned to one SMSP until finished

Thread



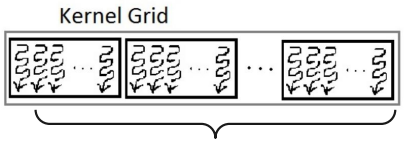


Executed on the execution lanes in

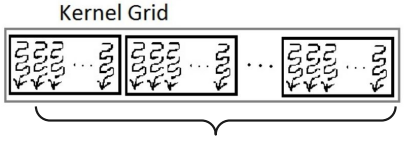
Programming Model vs GPU Hardware

Program	Created by	Resource-allocation	Hardware scheduler	Execution location
Kernel	CPU program through the CUDA runtime or driver	No fixed resources allocated	GPU command processor and thread-block scheduler	Multiple SMs
Grid	Defined by the kernel-launch configuration	Collection of pending and active blocks	Not directly scheduled	Multiple SMs
Thread block	Defined by the grid and block dimensions	Registers, shared memory, thread slots, warp slots, and block slots on one SM	Thread-block scheduler	Assigned to a single SM
Warp	Automatically formed from threads in a block	Warp slots and register state associated with an SMSP	Warp scheduler	Assigned to one SMSP
Thread	Defined by the programmer	Per-thread registers and logical execution state	Follows its warp	Execution lanes (CUDA Core, SFU, etc)

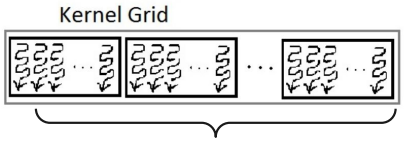
Single Kernel On GPU



90 Blocks



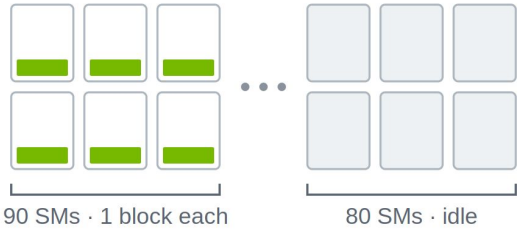
180 Blocks



4096 Blocks

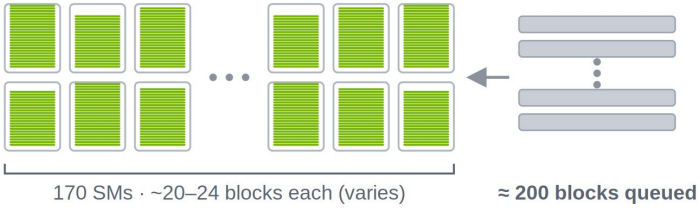


170 SMs



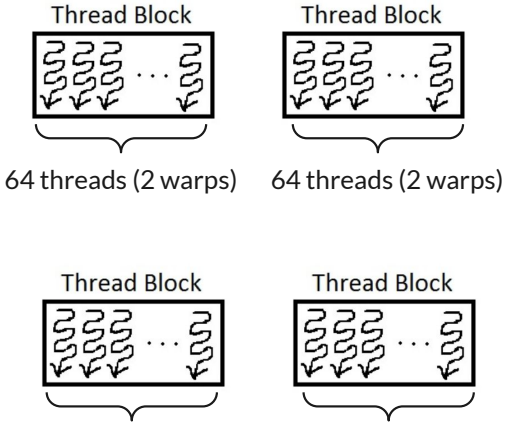


Multiple Blocks can operate on one SM if resource permits



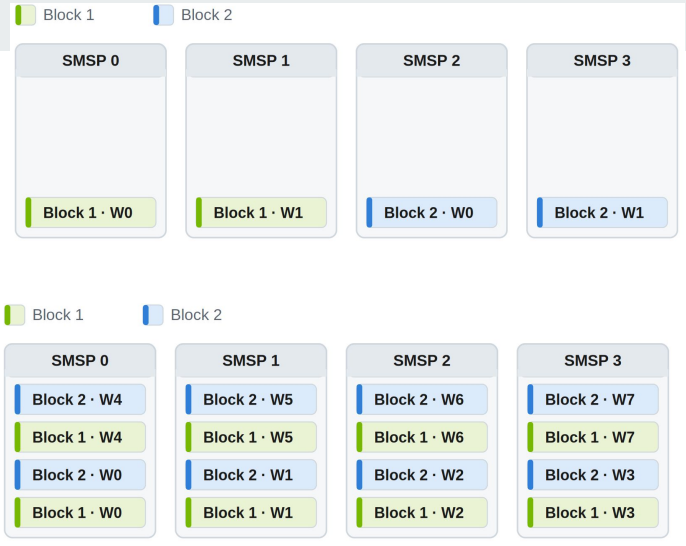
24 is the hardware limit for max #blocks on one SM

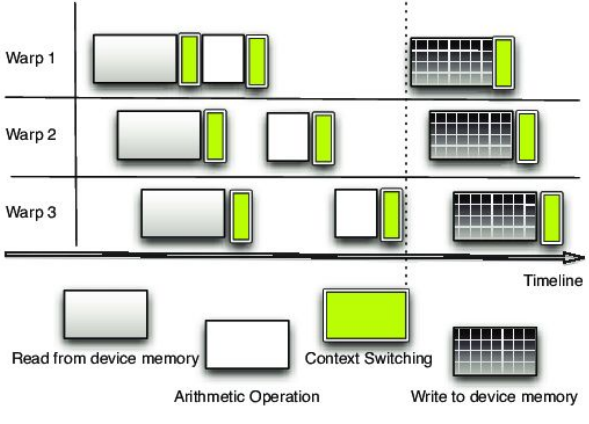
Block On SM

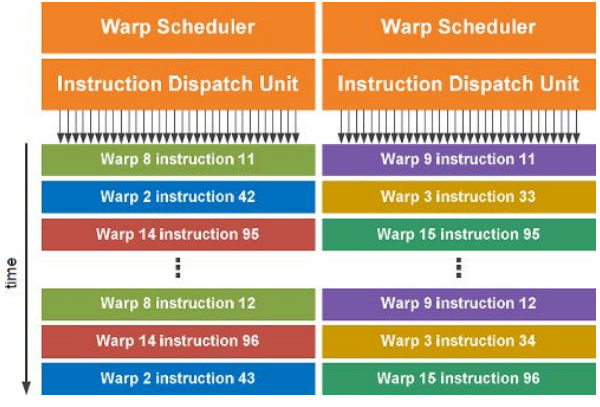




Each SM has **4 SMSP**, each of them has one **warp scheduler**

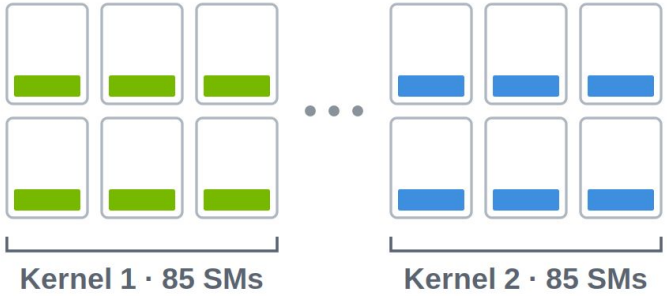




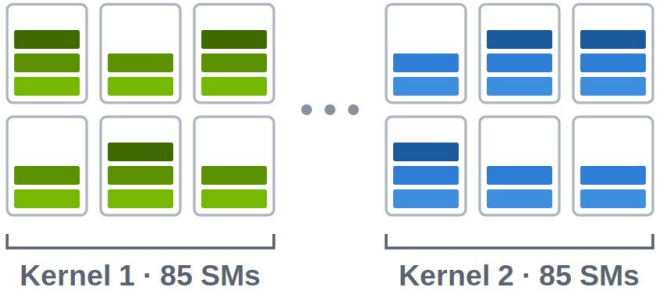


Colocation: Just give them separate SMs

Each kernel grid has < ½ * total SMs, or `CUDA_MPS_ACTIVE_THREAD_PERCENTAGE` is set to 50 for both.



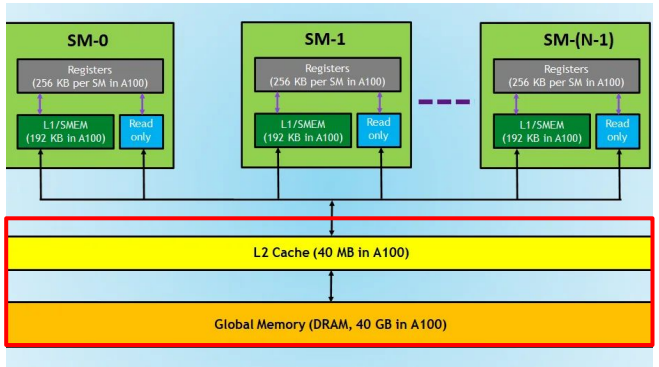
Case 1: Each kernel has only 85 blocks



Case 2: MPS Thread is set to 50 for both kernels

Does this fully isolate kernel 1 and kernel 2 from each other?

No



Experiment 1: Shared L2 Cache

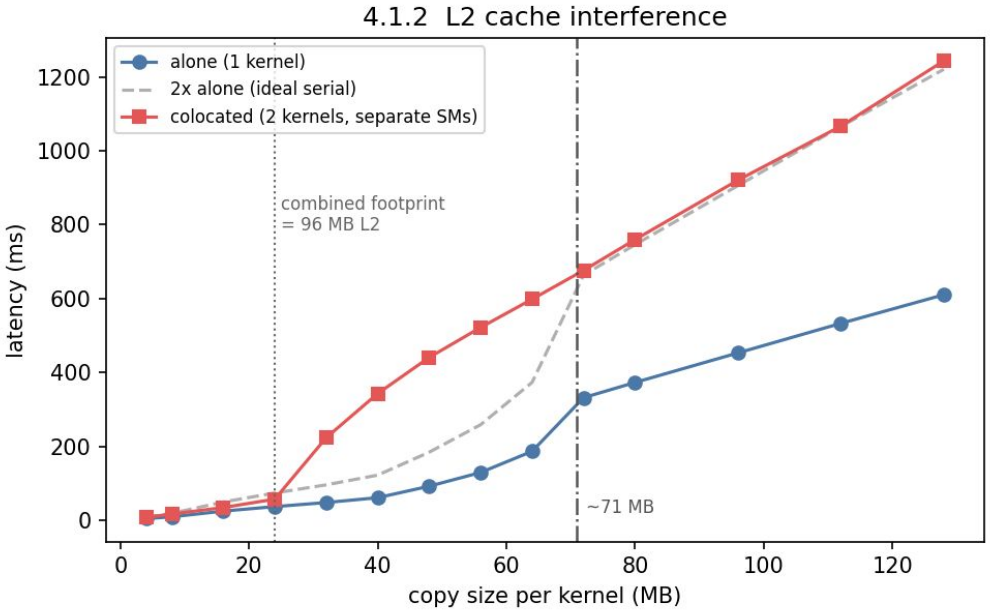
Copy kernel: every thread streams a slice of the array copying out[i] = in[i], and repeats num_itr times

```
__global__ void copy_kernel(float *in, float *out, long long num_floats, long long num_itr) {
    size_t start = threadIdx.x + blockDim.x * blockIdx.x;
    size_t step  = gridDim.x  * blockDim.x;          // grid stride
    for (size_t j = 0; j < num_itr; j++)             // repeat the whole copy num_itr times
        for (size_t i = start; i < num_floats; i += step)
            out[i] = in[i];                          // coalesced load + store
}
```

Two copy kernels are launched concurrently at 2 CUDA streams, each contains 85 Blocks (half of #SM), measuring latency compared to separate launch baseline.

```
# baseline: ONE kernel alone
record(start)
copy<<<85, 1024>>>(inA, outA, n, itr)
record(stop); synchronize(stop)
t_alone = elapsed(start, stop)
# colocated: the two kernels at once, on separate streams
record(start)
copy<<<85, 1024, streamA>>>(inA, outA, n, itr)   # kernel A  ┐ overlap on
copy<<<85, 1024, streamB>>>(inB, outB, n, itr)   # kernel B  ┘ separate streams
record(stop); synchronize(stop)
t_coloc = elapsed(start, stop)                   # makespan of the pair
```

Experiment 1: Shared L2 Cache

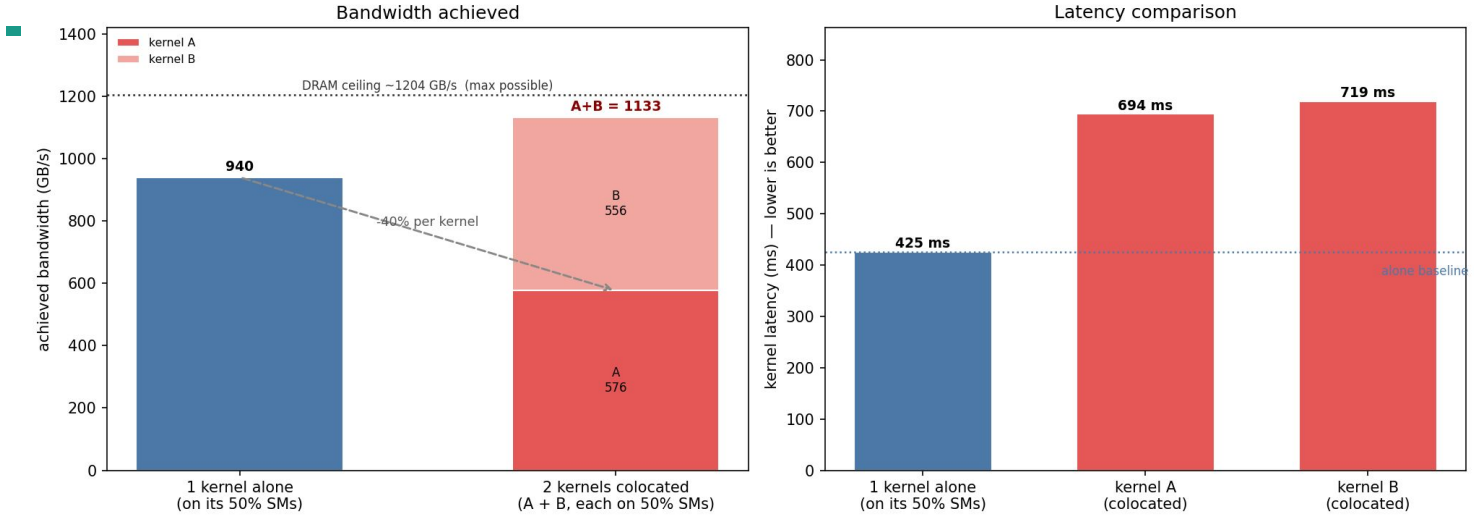


- Worst case: ≈ 5.6× a single kernel, and ≈ 2.8× the serial (2×-alone) time
- 24–71 MB: serial wins. Combined footprint (4× size) overflows L2 → the kernels evict each other's data.
- > 71 MB: back to 2× (serial). Even one kernel overflows L2; they just split DRAM bandwidth.

Takeaway: Even on separate SMs the kernels still share one L2 cache; performance falls off a cliff at L2 cache capacity.

Experiment 2: Shared DRAM Bandwidth

Still the copy kernel, but with a 4 GB array that overflows L2 cache. SM split is enforced with MPS



- One kernel alone on half the GPU already reaches 940 GB/s.
- Colocated, each kernel falls to ~566 GB/s (−40%); together they deliver 1133 GB/s, suppressed by 1204 DRAM ceiling.
- Same thing as latency: 425 ms alone → ~700 ms colocated, each kernel ~65% slower.

Colocation: SMs Sharing

In production, a GPU kernel almost never launches fewer blocks than the GPU has SMs.

For example, Llama-3-8B's up-projection of FFN is a matrix multiply:

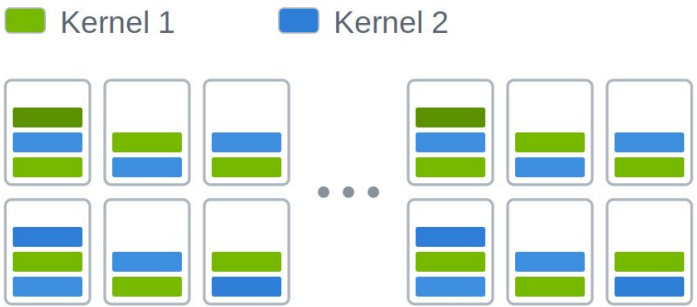
$$[\ \text{tokens} \times 4096\ ] \times [\ 4096 \times 14336\ ] \rightarrow [\ \text{tokens} \times 14336\ ].$$

For a prefill of 2048 tokens, cuBLAS dispatches a tensor-core GEMM kernel that tiles the 2048 × 14336 output into 128×128 tiles:

$$\frac{2048}{128} \times \frac{14336}{128} = 16 \times 112 = \mathbf{1{,}792\ blocks}.$$

That is more than 10× the SMs on the GPU (170 on a 5090, 132 on a H100)

Therefore, kernel colocation is only practical in the scenario of SM sharing:



How to share an SM



Important Resources:

- 1536 threads at most.
- 4 warp schedulers
- 256KB L1 Data Cache
- Accelerators:
  - SFU
  - Tensor Coares
  - Tensor memory accelerators

Experiment 3: Thread Capacity

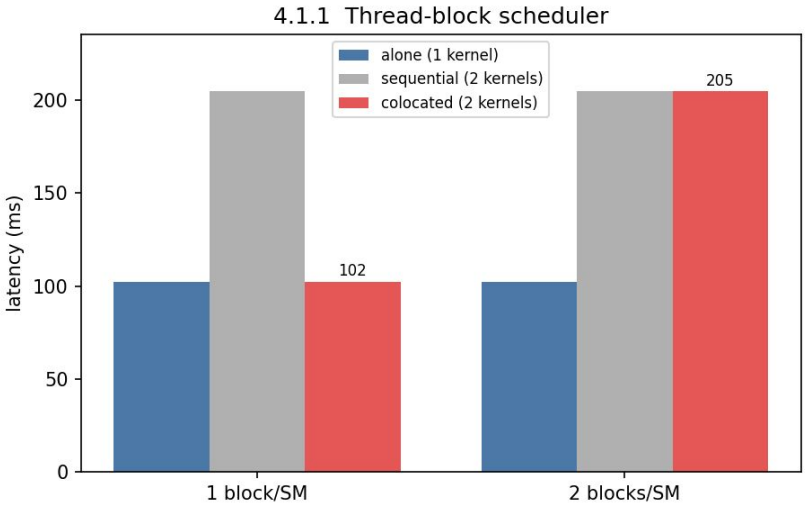
Sleep kernel: repeatedly sleeps ~1 μs with num_itr = 100000; no memory use.

Launch two sleep kernel concurrently,

each has 170 or 340 block, each block has 768 threads (half the max threads per SM)

```
__global__ void sleep_kernel(long long num_itr) {
    for (long long i = 0; i < num_itr; ++i)
        asm volatile("nanosleep.u32 1000;");
}
```

```
threads = 768                    # half the 1536-thread SM budget
for blocks in [170, 340]:        # 768, 1536 threads
    record(start)
    sleep<<<blocks, threads, streamA>>>(itr)   # kernel A  ┐ overlap on
    sleep<<<blocks, threads, streamB>>>(itr)   # kernel B  ┘ separate streams
    record(stop); synchronize(stop)
    t_coloc = elapsed(start, stop)             # makespan of the pair
```



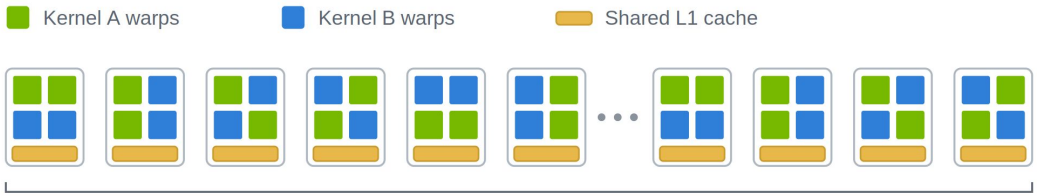
Takeaway: Once each kernel already fills an SM's thread slots, colocation gives zero overlap

Experiment 4: Shared L1 Cache

Block Copy kernel (`copy_kernel_per_tb`**)**: similar to the copy kernel but each block gets its own data to copy.

Setup: Two block copy kernels are launched concurrently with the following setup

	Block Copy kernel A	Block Copy kernel B
Launched from	stream 1	stream 2
Blocks per kernel	170	170
SMs each kernel uses	all 170	all 170
Threads per block	64 (2 warps)	64 (2 warps)
SMSPs used per SM	2 of 4	the other 2 of 4



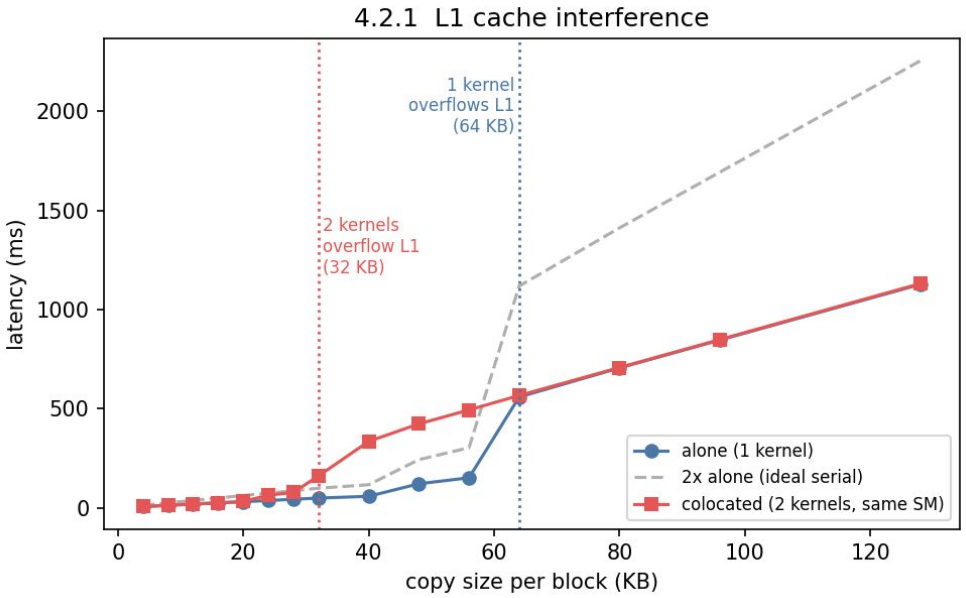
170 SMs, each has 4 SMSP and one 128kb shared L1 cache

Baseline: one Block Copy kernel alone (its latency t_alone). (2 blocks per SM)

Varied: the copy size per block (4 KB → 128 KB),

Measured: the colocated latency of the two kernels, compared against t_alone and against 2 × t_alone (the serial case).

Experiment 4: Shared L1 Cache



Alone case: 2 Blocks per SM

Colocation: 4 Blocks per SM

- 32 → 64 KB: serial wins. The two blocks' combined footprint overflows the 128 KB L1
- > 64 KB: back to 2× (serial). Even one kernel alone overflows L1, so there is no reuse left to lose.
- if both cannot fit in L1 cache, falling off to L2 cache gives benefit to colocation.

Takeaway: the same capacity cliff as the L2 cache, but past that point the data no longer fits in L1, colocation regains its concurrency gain

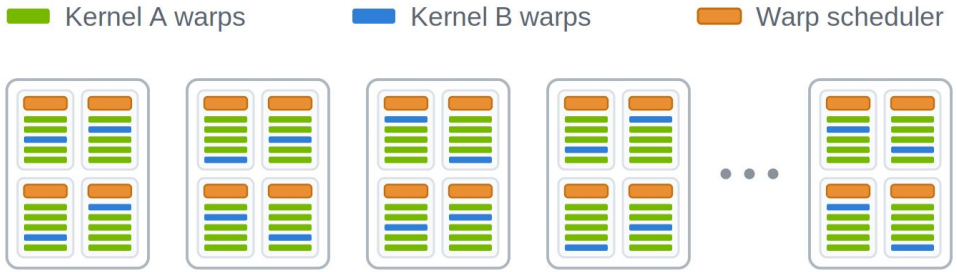
Experiment 5: Warp Scheduler

A natural intuition: a **compute-bound kernel** and a **memory-bound kernel** complement each other, so colocating them should be an easy win, since they stress different resources of the GPU.

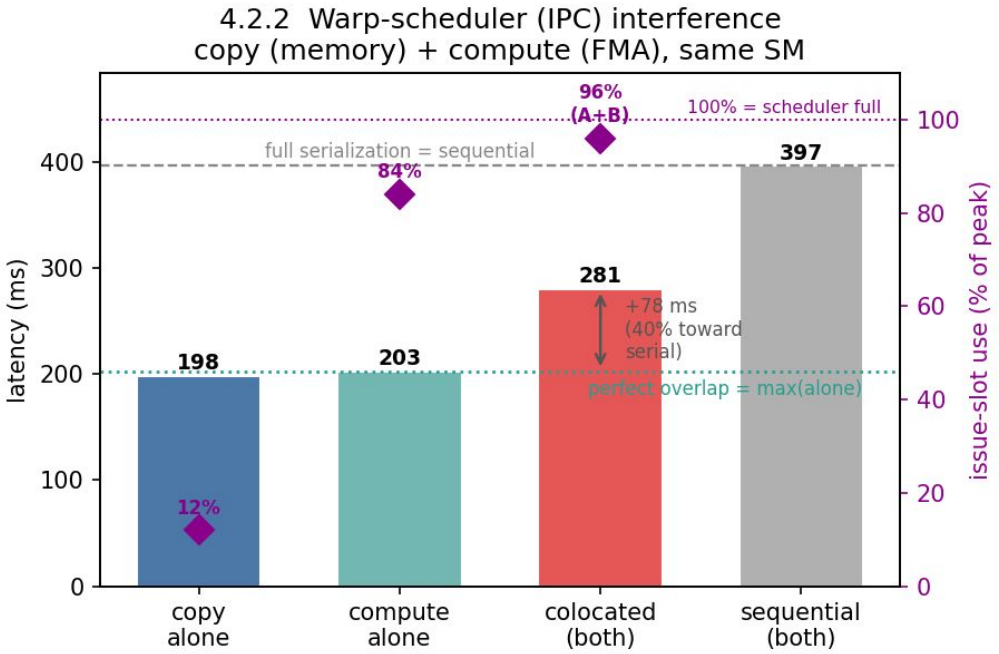
In the SM sharing scenario, this is not always the case, especially when they share the same SMSP, which is normal in large kernels.

Setup: large memory bound kernel and small compute bound kernels as the table shows.

	Kernel A: Block copy	Kernel B — FMA compute
Kernel	`copy_kernel`, 16 MB array	`mul_fp32_ilp4`
Launched from	stream 1	stream 2
Blocks per kernel	170 (one per SM)	170 (one per SM)
Threads per block	512 (16 warps)	128 (4 warps)



Experiment 5: Warp Scheduler



Why not fully overlap:

every instruction, a memory load or a compute FMA, must first be issued by the SM's warp schedulers, which can issue **only 4 per cycle.**

The compute kernel alone already keeps the scheduler ~83% busy, so adding the copy's loads runs it out of issue slots

The scheduler can't feed both kernels every cycle, so their warps wait their turn and both slow down.

What is measurable for arbitrary kernels

The aforementioned kernels and workloads are crafted to produce interference intentionally

For arbitrary CUDA Kernel, how much metrics can we profile to predict interference?

Simple profiler using Nsight Compute

column	NCU counter	what it really measures	what it cannot measure
SM occupancy	`sm__warps_active`	warp-slot **residency**: avg % of the 48 warp slots/SM holding a warp	whether those warps make *progress* — a stalled warp counts the same as a running one
DRAM bandwidth	`gpu__dram_throughput`	achieved DRAM transfer **rate**, % of peak GB/s	latency sensitivity; unique-bytes *footprint* (traffic ≠ working set)
L2 cache	`lts__throughput`	L2 **bandwidth**: sectors served per cycle, % of peak	L2 **space held** — capacity/footprint, i.e. combined-working-set thrash
L1 cache	`l1tex__throughput`	L1TEX **bandwidth**: requests served per cycle, % of peak (NOT hit rate, NOT bytes resident)	L1 **space held** — no counter reports cache occupancy; footprint only *inferred* (hit rate + cold-miss bytes, below)
Warp scheduler	`sm__inst_issued`	issue-slot **rate**: instructions issued vs max 4/cycle/SMSP	*promptness* — whether a just-ready warp gets picked now; starvation of a latency-bound co-tenant
FP32 FMA pipe	`sm__inst_executed_pipe_fma`	FP32-FMA pipe issue **rate**, % of pipe peak	datapath shared with FP64 on consumer parts — the two pipe counters pretend independence
FP64 pipe	`sm__inst_executed_pipe_fp64`	FP64 pipe issue **rate**, % of pipe peak	same shared-datapath coupling with FP32-FMA

What is measurable for arbitrary kernels

Result:

kernel	SM occupancy	DRAM bandwidth	L2 cache	L1 cache	Warp scheduler	FP32 FMA pipe	FP64 pipe
`sleep`	50.0%	0.0%	0.0%	0.0%	0.6%	0.0%	0.0%
`dram`	33.2%	69.0%	27.3%	13.9%	3.2%	0.4%	0.0%
`l2`	24.5%	0.1%	76.3%	39.6%	12.2%	1.3%	0.0%
`l1`	4.2%	0.0%	46.1%	23.6%	5.3%	1.5%	0.0%
`fma`	8.3%	0.0%	0.0%	0.0%	82.7%	79.0%	0.0%
`fp64`	8.3%	0.0%	0.0%	0.0%	7.7%	0.0%	99.5%

What is measurable for arbitrary kernels

Predicted interference:

interfere? ↓ \ with →	sleep	dram	l2	l1	fma	fp64
sleep	No	No	No	No	No	No
dram	No	Yes	Yes	No	No	No
l2	No	Yes	Yes	Yes	No	No
l1	No	No	Yes	No	No	No
fma	No	No	No	No	Yes	No
fp64	No	No	No	No	No	Yes

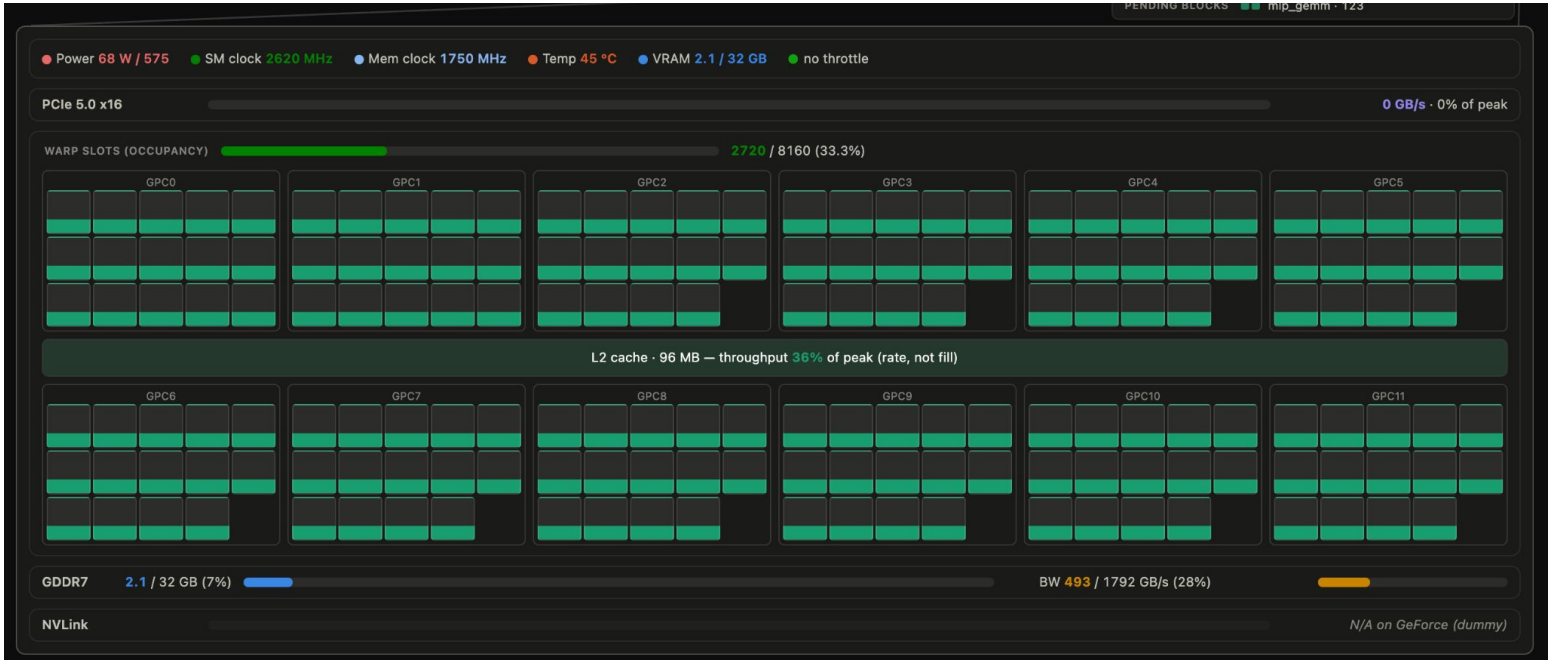
Validated Interference:

measured ↓ \ with →	sleep	dram	l2	l1	fma	fp64
sleep	1.00×	1.00×	1.00×	1.00×	1.00×	1.00×
dram	1.00×	1.98×	1.07×	1.01×	1.01×	1.00×
l2	0.98×	1.40×	2.01×	1.28×	1.59×	1.25×
l1	0.92×	0.92×	0.91×	2.50×	1.87×	0.95×
fma	0.91×	0.91×	1.57×	0.97×	1.60×	0.93×
fp64	0.91×	0.91×	0.91×	0.98×	2.28×	1.80×

- Cache occupancy is not measurable, what measured is bandwidth usage only.
- Kernels that nearly saturate warp scheduler (FMA, 83%) can cause starvation for other kernels regardlessly even when sum is under 100%

What is measurable for arbitrary workload

Demo: Visualized GPU utilization in SM level, profiling a **llama3-8B** workload during one prefill and four decode, implemented with CUPTI and NVML.



Interference-Aware Scheduling

Requirements for a scheduler that colocates workloads with performance guarantees.

- **Granularity**: decide per-kernel, not per-whole-workload to avoid head-of-line blocking due to resource demanding kernels.
- **Predict interference**: requires kernel-level interference estimator from each kernel's per-resource demand, then aggregate to workload level.
- **Performance vs. efficiency**: balance between minimizing interference and maximizing GPU utilization.
- **GPU architecture awareness**: cuDNN/cuBLAS behave differently across architectures, so reprofile is required on each new GPU

Interference-Aware Kernel

CUDA kernels are usually built to run alone, allocating all the SMs, registers, and shared memory, without colocation in mind.

But they may **not require all resources they claimed**. In the llama3-8B case we shown,

If forced to use only 50% of SMs:

Decode (memory-bound) → ~10% slowdown

Prefill (compute-bound) → 70% slowdown

Takeaway: Marginal performance gains may come at the cost of GPU utilization and overall efficiency.

A scheduler can improve efficiency by replacing default kernels with colocation aware variants at launch time.